

In[68]:=

Assignment 04: Exploring Spiking Neurons with Wolfram

Name: Brandie Griffin

Course: ITAI 4374 – Neuroscience as a Model for AI

Instructor: Patricia McManus
Houston Community College

Overview

This was also my first time using Wolfram Language, so I had to troubleshoot several simulation errors while learning how NDSolve and event detection work. This notebook explores the behavior of spiking neurons using the Leaky Integrate-and-Fire (LIF) model implemented in Wolfram Language. The assignment demonstrates solving the LIF differential equation using NDSolve, visualizing neuron activity, creating an interactive dashboard using Manipulate, and generating a firing-rate curve that shows how spike frequency changes with input current.

```
In[69]:= Print["Wolfram Language ready!"]  
Print[$Version]  
  
Wolfram Language ready!  
14.3.0 for Linux x86 (64-bit) (July 8, 2025)
```

Part A – LIF Neuron Setup

Define Neuron Parameters

```
In[71]:= (* ===== Neuron Parameters ===== *)
```

```
In[72]:=
```

```
tau = 0.020;
Vrest = -70.0;
Vthresh = -55.0;
Vreset = -75.0;
R = 1.0;
```

Define Input Current

```
In[77]:= (* ===== Input Current ===== *)
```

```
Iinput[t_] := If[0.05 ≤ t ≤ 0.4, 18.0, 0.0];
Print["Parameters defined."]
```

```
Parameters defined.
```

Knowledge Check

At $t = 0.03$ the input current returns 0 because the current is only active between 0.05 and 0.4 seconds.

At $t = 0.2$ the function returns 18.0 because the time falls inside the active current window.

At $t = 0.5$ the function again returns 0 because the input current has already turned off.

Delayed assignment ($:=$) is used so that the function evaluates the value of t each time it runs. If immediate assignment ($=$) were used, the function would evaluate only once and would not update properly for different time values.

Verify Input Current Behavior (values)

```
In[79]:= Print["Iinput[0.03] = ", Iinput[0.03]]
```

```
Print["Iinput[0.20] = ", Iinput[0.20]]
```

```
Print["Iinput[0.50] = ", Iinput[0.50]]
```

```
Iinput[0.03] = 0.
```

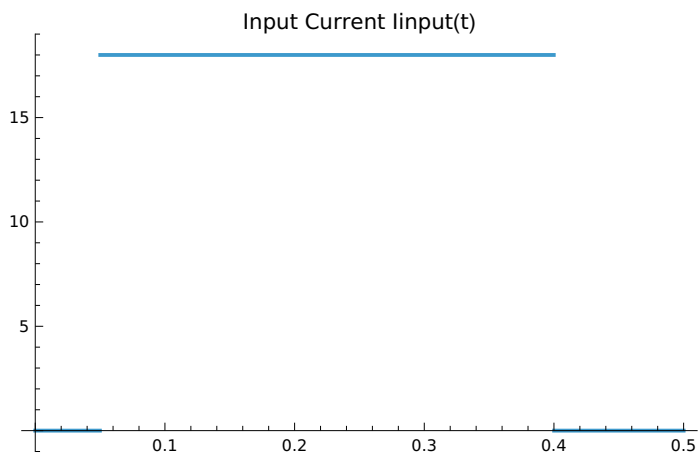
```
Iinput[0.20] = 18.
```

```
Iinput[0.50] = 0.
```

Verify Input Current Behavior (plot)

```
In[82]:= Plot[Iinput[t], {t, 0, 0.5}, PlotLabel -> "Input Current Iinput(t)"]
```

Out[82]=



Part B – Interactive Neuron Dashboard

```

In[83]:= Manipulate[
  Module[{Vm, tt, sol, spikeTimes},

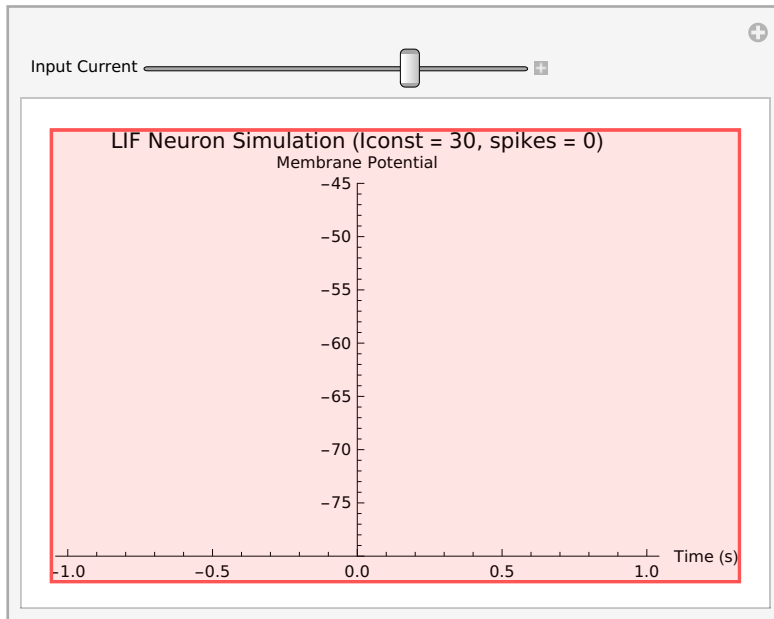
    {sol, spikeTimes} =
      Reap[
        NDSolveValue[
          {
            tau*Vm'[tt] == -(Vm[tt] - Vrest) + R*Iconst,
            Vm[0] == Vrest,
            WhenEvent[Vm[tt] ≥ Vthresh, {Sow[tt], Vm[tt] → Vreset}]
          },
          Vm,
          {tt, 0, 0.5},
          MaxStepSize → 0.0005
        ]
      ];

    spikeTimes = If[Length[spikeTimes] > 0, Flatten[spikeTimes], {}];

    Plot[
      sol[tt],
      {tt, 0, 0.5},
      PlotRange → {-80, -45},
      PlotPoints → 300,
      Exclusions → None,
      AxesOrigin → {0, -80},
      AxesLabel → {"Time (s)", "Membrane Potential"},
      PlotLabel → Row[{
        "LIF Neuron Simulation (Iconst = ", Iconst,
        ", spikes = ", Length[spikeTimes], ")
      }],
      Epilog → {Red, Dashed, Line[{{0, Vthresh}, {0.5, Vthresh}}]}
    ]
  ],
  {{Iconst, 30, "Input Current"}, 5, 40, 1}
]

```

Out[83]=



Example Output (Static Simulation)

```
In[84]:= (* Static example for PDF export *)
```

```
Iconst = 30;
```

```
Module[{Vm, tt, sol, spikeTimes},
```

```
{sol, spikeTimes} =
```

```
Reap[
```

```
NDSolveValue[
```

```
{
```

```
tau*Vm'[tt] == -(Vm[tt] - Vrest) + R*Iconst,
```

```
Vm[0] == Vrest,
```

```
WhenEvent[Vm[tt] ≥ Vthresh, {Sow[tt], Vm[tt] → Vreset}]
```

```
},
```

```
Vm,
```

```
{tt, 0, 0.5},
```

```
MaxStepSize → 0.0005
```

```
]
```

```
];
```

```
spikeTimes = Flatten[spikeTimes];
```

```
Print["Neuron fired ", Length[spikeTimes], " spikes"];
```

```
Plot[
```

```
sol[tt],
```

```
{tt, 0, 0.5},
```

```
PlotRange → {-80, -45},
```

```
AxesLabel → {"Time (s)", "Membrane Potential"},
```

```
PlotLabel → "Example Neuron Response (Iconst = 30)",
```

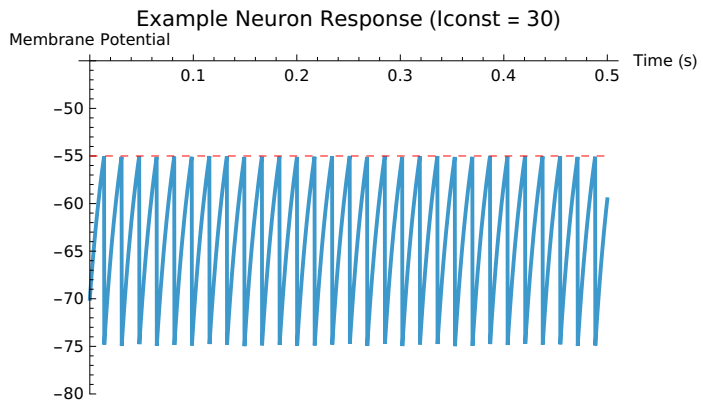
```
Epilog → {Red, Dashed, Line[{{0, Vthresh}, {0.5, Vthresh}}]}
```

```
]
```

```
]
```

```
Neuron fired 29 spikes
```

Out[85]=



Part C – f-I Curve

```
In[86]:= currents = Range[5, 40, 2];
```

```

rates =
Table[
Module[{Vm, tt, sol, spikes},

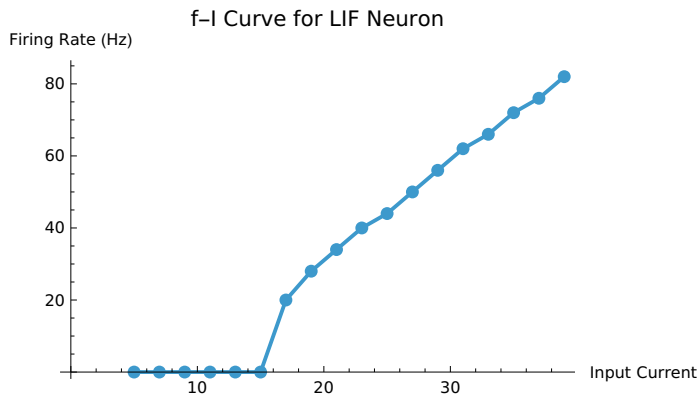
{sol, spikes} =
Reap[
NDSolveValue[
{
tau*Vm'[tt] == -(Vm[tt] - Vrest) + R*I,
Vm[0] == Vrest,
WhenEvent[Vm[tt] ≥ Vthresh, {Sow[tt], Vm[tt] → Vreset}]
},
Vm,
{tt, 0, 0.5},
MaxStepSize → 0.0005
]
];

Length[Flatten[spikes]]/0.5
],
{I, currents}
];

ListLinePlot[
Transpose[{currents, rates}],
AxesLabel → {"Input Current", "Firing Rate (Hz)"},
PlotMarkers → Automatic,
PlotLabel → "f-I Curve for LIF Neuron"
]

```


Out[88]=



Knowledge Check

The rheobase current is the smallest input current where the neuron begins to fire spikes. From the f-I curve, the neuron begins firing at approximately 15 nA, which represents the minimum current required for the membrane potential to reach the firing threshold.

The f-I curve shows that the firing rate increases as the input current increases. Once the neuron begins firing, the relationship between input current and firing rate becomes roughly linear. This means that stronger input currents cause the neuron to spike more frequently within the same time window.

In this simplified LIF neuron model, the firing rate continues to increase as current increases. In real biological neurons, the firing rate may eventually saturate due to biological limitations, but the LIF model mainly demonstrates how spike frequency increases with stronger input signals.

When the input current increases, the membrane potential reaches the threshold faster, which results in spikes occurring more frequently. If the current is too weak, the membrane potential never reaches the threshold and the neuron does not fire.

Part D – Sine Wave Encoding

In[89]:= `Isine[t_] := 15.0 + 12.0*Sin[2*Pi*5*t]`

```

In[90]:= Module[{Vm, tt, sol, spikeTimes, inputPlot, voltagePlot},

  {sol, spikeTimes} =
    Reap[
      NDSolveValue[
        {
          Vm'[tt] == -(Vm[tt] - Vrest) + R*Isine[tt]/tau,
          Vm[0] == Vrest,
          WhenEvent[Vm[tt] ≥ Vthresh, {Sow[tt], Vm[tt] → Vreset}]
        },
        Vm,
        {tt, 0, 1.0}
      ]
    ];

  spikeTimes = Flatten[spikeTimes];

  Print["Neuron fired ", Length[spikeTimes], " spikes (sine-wave input)"];
  Print["Spike times: ", spikeTimes];

  voltagePlot = Plot[
    sol[tt],
    {tt, 0, 1.0},
    PlotRange → {-80, -45},
    AxesLabel → {"Time (s)", "Membrane Potential"},
    PlotLabel → "Neuron Response to Sine-Wave Input",
    Epilog → {Red, Dashed, Line[{0, Vthresh}, {1.0, Vthresh}]}
  ];

  inputPlot = Plot[
    Isine[t],
    {t, 0, 1.0},
    AxesLabel → {"Time (s)", "Input Current"},
    PlotLabel → "Sine-Wave Input Current"
  ];

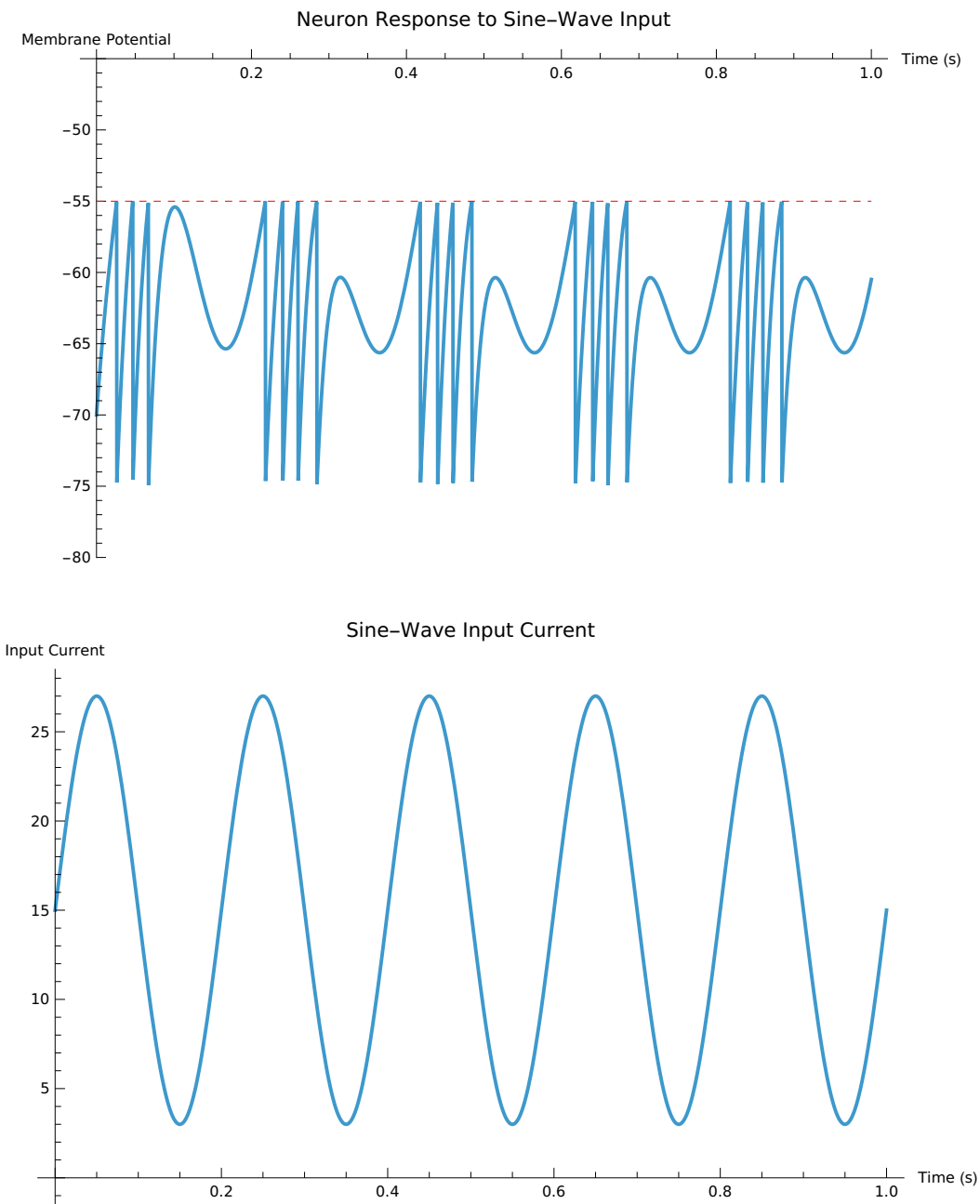
  GraphicsColumn[{voltagePlot, inputPlot}]
]

```

Neuron fired 19 spikes (sine-wave input)

Spike times: {0.0260992, 0.0468183, 0.0671122, 0.217907, 0.240209, 0.260038, 0.284442, 0.41797, 0.440257, 0.460087, 0.484523, 0.617971, 0.640257, 0.660087, 0.684524, 0.817971, 0.840257, 0.860087, 0.884524}

Out[90]=



Reflection

The sine-wave input shows how a continuous analog signal can be converted into discrete spike events by the neuron model. When the sine wave increases, the membrane potential rises and

reaches the firing threshold more easily, which produces more spikes. When the sine wave decreases, the membrane potential drops and the neuron fires less frequently or stops firing. This demonstrates how neurons encode information from continuous signals using spike timing and spike frequency rather than preserving the exact shape of the original signal. This demonstrates how neurons encode information through spike frequency and spike timing rather than transmitting the exact analog signal itself.

Experiment

Experiment 1 – Increasing Tau

Parameter change: In this experiment the membrane time constant τ was increased from 0.020 to 0.040.

Observation: After increasing τ , the membrane potential rises more slowly. Because the neuron integrates input more gradually, the spike timing becomes slightly delayed compared to the original simulation.

Explanation: The parameter τ controls how quickly the membrane potential responds to input current. A larger τ means the neuron integrates incoming signals over a longer time period, which slows the rate of change of the membrane voltage. Because of this slower integration, the membrane potential rises more gradually and spikes occur later compared to the original simulation.

Conclusion: This experiment shows that τ affects how quickly the neuron can respond to incoming signals. Larger values make the neuron respond more slowly but also allow it to accumulate input over time.

Experiment 2 – Increasing Threshold

Parameter change: The spike threshold was increased from -55 mV to -50 mV.

Observation: After increasing the threshold, the neuron fired fewer spikes because the membrane potential had to rise higher before a spike could occur.

Explanation: The threshold determines the voltage level required to trigger a spike. When the threshold increases, the neuron needs stronger input to reach that level.

Conclusion: Increasing the threshold reduces spike frequency because it becomes harder for the neuron to reach the firing condition.

Experiment 3 – Increasing Input Current

Parameter change: The input current amplitude was increased.

Observation: As the input current increased, the neuron fired more spikes within the same time window.

Explanation: A stronger input current pushes the membrane potential toward the threshold faster, causing spikes to occur more frequently.

Conclusion: This experiment shows that input current strength directly influences spike frequency, which is one of the main ways neurons encode information.

Part F – Final Reflection Questions

1. What did you learn about how spiking neurons work from this simulation?

Working through this assignment helped me understand how spiking neurons convert input signals into discrete spike events. Instead of continuously outputting a value like many traditional models, the neuron only fires when the membrane potential reaches a certain threshold. Once that threshold is reached, the neuron resets and begins integrating input again. Seeing how the membrane potential changes over time made it easier to understand how biological neurons process information through spike timing and spike frequency.

2. How does input current influence firing behavior?

The input current directly affects how quickly the membrane potential reaches the firing threshold. When the input current is low, the membrane potential may never reach the threshold and no spikes occur. As the input current increases, the membrane potential rises faster, which causes the neuron to fire more frequently. This relationship was clearly visible in the f-I curve where higher current values resulted in more spikes within the same time window.

3. What was the most challenging part of this assignment?

The most challenging part of this assignment was troubleshooting errors while learning to use Wolfram for the first time. At one point the simulation stopped working because the ClearAll command removed previously defined parameters like Vrest. It took some time to realize that the parameters needed to be redefined before running the simulation again. Another challenge was making sure the spike events were recorded correctly during the simulations.

4. What debugging steps did you take to solve problems in the notebook?

To solve these issues, I carefully checked which variables were defined and reran the parameter cells when necessary. I also reviewed error messages from Wolfram to understand what the system was telling me. Breaking the notebook into smaller sections and testing each part individually helped identify where the problems were occurring. Once the variables and simulation steps were running in the correct order, the model began producing the expected spike results.

5. How does this simulation relate to real biological neurons?

Although this model is simplified, it demonstrates important principles of how real neurons behave. Biological neurons also integrate incoming signals and fire spikes when their membrane potential crosses a threshold. The timing and frequency of these spikes allow neurons to encode information about stimuli. While real neurons involve more complex dynamics, the leaky integrate-and-fire model provides a useful approximation for studying how neural signals are generated and processed.